Software Engineering

18IS55

Syllabus-unit-I

- **Overview: Introduction:**
- *Professional Software Development, Software Engineering Ethics, Case studies.*
- **Software Processes:** *Models, Process activities, Coping with Change, Process improvement. The Rational Unified Process. Computer Aided Software Engineering. Agile Software Development: Introduction to agile methods, Agile development techniques, Agile project management and scaling agile methods.*

Unit-II

- **Requirements Engineering and System Modeling:**

Software Requirements: Functional and Non-functional requirements. Requirements Elicitation, Specification, Validation and Change. System Modeling: Context models, Interaction models, Structural models, Behavioural models, Model driven architecture. Architectural Design: Design decisions, Architectural views, Architectural patterns and architectures.

Unit-III

- **Development and Testing:**
- **Design and implementation:** *Object oriented design using UML, Design patterns, Implementation issues, Open-source development. Software Testing: Development testing, Test-driven development, Release testing, User testing.*
- **Software Evolution:** *Evolution processes. Legacy system evolution, Software maintenance*

Unit-IV

- **Advanced Software Engineering:**
- Dependable systems: *Dependability properties, Sociotechnical systems, dependable processes, formal methods and dependability, Reliability engineering: Availability and reliability, reliability requirements, Reliability measurements, Component based software engineering: Components and component models, CBSE processes, component composition.*

Unit-V

- **Software Management:**
- Project Management: *Risk Management, Managing People, Teamwork, Project Planning: Software Pricing, Plan driven development, Project Scheduling, Agile planning, Estimation Techniques, COCOMO cost modeling*.

Text Books

- Ian Sommerville,“ Software Engineering”, 9th Edition, Pearson Education, 2013, ISBN: 9788131762165
- Roger.S.Pressman,“ Software Engineering-A Practitioners Approach”, 7th Edition, Tata McGraw Hill, 2007, ISBN: 9780071267823

Lab-Part A

1) Identifying the Requirements from Problem Statements
2) Estimation of Project Metrics
3) Modeling UML Use Case Diagrams and Capturing Use Case Scenarios
4) Identifying Domain Classes from the Problem Statements
5) State chart and Activity Modeling
6) Modeling UML Class Diagrams and Sequence diagrams
7) Modeling Data Flow Diagrams
8) Estimation of Test Coverage Metrics and Structural Complexity
9) Designing Test Suites

Lab-Part B

- Student will analyse, design, and implement an application using the appropriate Software engineering tools and practices. All topics learnt in virtual lab (SE phases) need to be covered. A report of the same is expected to be submitted

Course Outcomes

- Co1: Comprehend various software life cycle models and steps of software development process.
- Co2: Apply concepts of Software Project Planning and software Design techniques
- Co3:Analyze capabilities of various tools to assist in the software development activities
- Co4: Develop robust software design and software project plan from requirement gathering to implementation

UNIT-I-Chapter 1,2,3

Professional software development

Software engineering ethics

Case studies

Objectives

- understand what software engineering is and why it is important;
- understand that the development of different types of software systems may require different software engineering techniques;
- understand some ethical and professional issues that are important for software engineers;

Software Engineering

- Deals with cost effective Software Development
- Systematic collection of past experience resulted in techniques, methodologies, guidelines for software development

Engineering approach of developing software:

- Past experience is used
- Systematically arranged
- Quantitative techniques provided
- Few thumb rules
- Pragmatic approach for cost effectiveness

Software Products

- **Generic products**
- **Customized products**

Software Products

- **Generic products :** standalone systems that are manufactured and marketed, sold to any customer who wants to use them
- **Ex:** graphics programs, appointment booking software, project management tools, CAD software.

- **Customized products:** software made to a specific customer to meet their own needs.
- **Ex:** air traffic control software, traffic control software, banking software.

Software Products

- Specifications of software, changes decisions are made by developer in case of generic software

- Specifications , changes decision are made by customer who owns the software.

FAQs about software engineering

- What is software
- What are the attributes of good software
- What is software engineering
- What are the fundamental software engineering activities
- What is the difference between software engineering and computer science
- What is the difference between software engineering and system engineering

FAQs about software engineering

- Key challenges facing software engineering
- Costs of software engineering
- Best software engineering techniques and methods

Attributes of a good software

- Maintainability
- Dependability and security
- Efficiency
- Acceptability

Software process activities

- Software specification
- Software development
- Software validation
- Software evolution

Types of application in determining SE methods

- Stand-alone
- Interactive transaction-based applications
- Embedded control systems
- Batch processing systems
- Entertainment systems
- System for modeling and simulation
- Data collection system

General issues that affect software

- Heterogeneity
- Business and social change
- Security and trust
- Scale

Software engineering ethics

- **Confidentiality:**Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- **Competence:** Engineers should not misrepresent their level of competence. They should not knowingly accept work which is out with their competence
- **Intellectual property rights**:Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected

Software engineering ethics

- **Computer misuse:**Software engineers should not use their technical skills to misuse other people's computers.

Case studies

- A personal insulin pump
- A mental health case patient management system
- A wilderness weather station
- iLearn: a digital learning environment

Software Processes

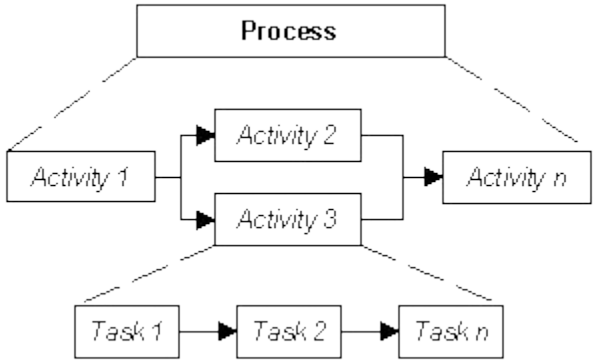
- understand the concepts of software processes and software process models;
- introduced to three generic software process models and when they might be used
- know about the fundamental process activities of software requirements engineering, software development, testing, and evolution;
- understand why processes should be organized to cope with changes in the software requirements and design;
- understand how the Rational Unified Process integrates good software engineering practice to create adaptable software processes.

Topics

- Software process models
- Process activities
- Coping with change
- Process improvement

Basic Definition

- Task: basic unit of work, well defined, assigned to one person
- Activity: group of related task in software development, each activity has input and output
- Process: set of related activities that leads to final software.



Software process activities

- Specification – defining what the system should do;
- Design and implementation – defining the organization of the system and implementing the system;
- Validation – checking that it does what the customer wants;
- Evolution – changing the system in response to changing customer needs

- A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective.

Process descriptions may also include:

- Products, which are the outcomes of a process activity;
- Roles, which reflect the responsibilities of the people involved in the process;
- Pre- and post-conditions, which are statements that are true before and after a process activity has been enacted or a product produced.

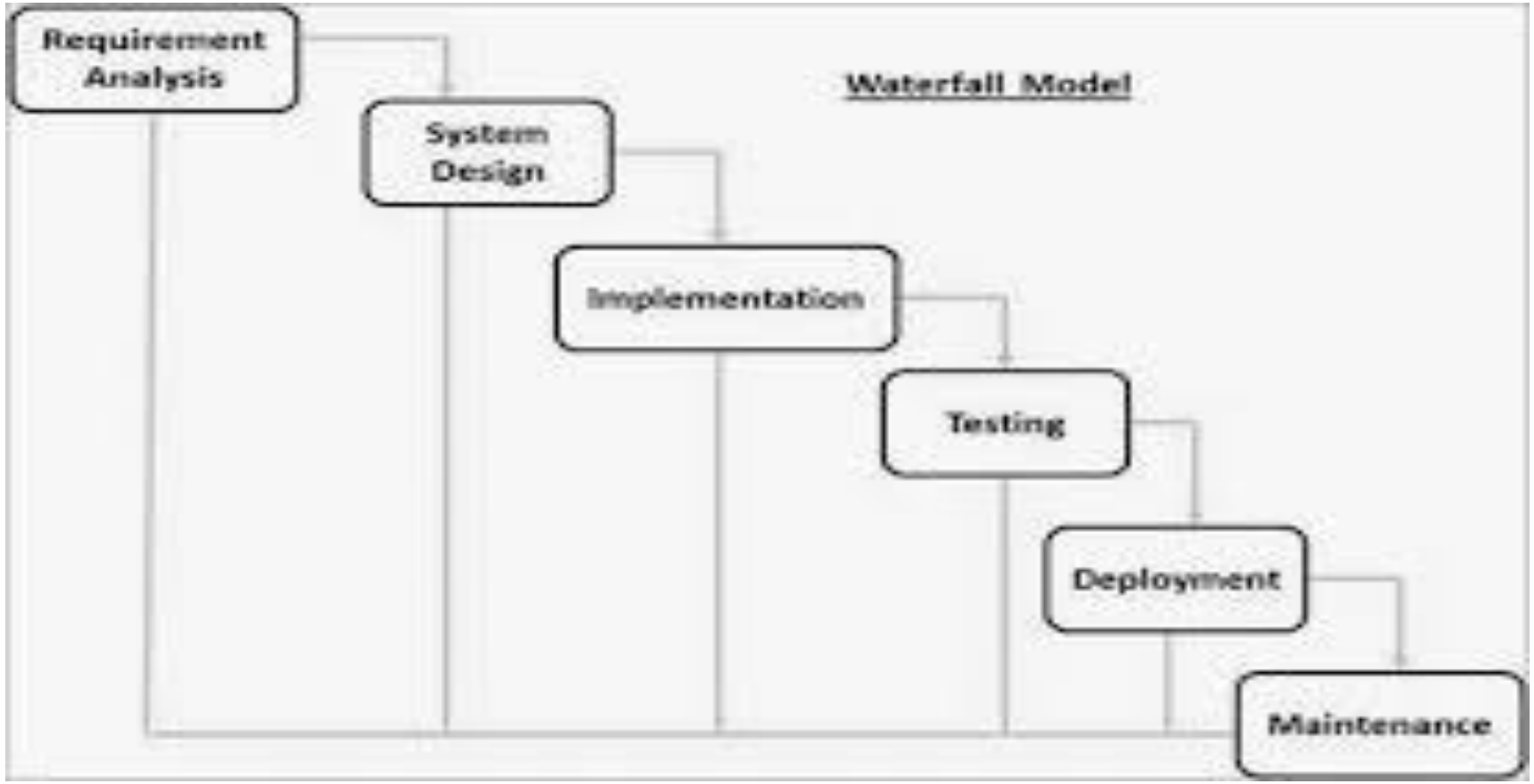
Plan-driven and agile processes

- Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan.
- In agile processes, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- In practice, most practical processes include elements of both plan-driven and agile approaches.
- There are no right or wrong software processes

Software process models

- The waterfall model
- Incremental development
- Reuse oriented Software Engineering

Water fall model



Water fall model

- There are separate identified phases in the waterfall model:

- Requirements analysis and definition
- System and software design
- Implementation and unit testing
- Integration and system testing
- Operation and maintenance

Water fall model

- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway.
- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements
- The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites

Water fall model-example-Banking Application

- Requirements are gathered by the business analyst and they are analyzed by the team. Requirements are documented during this phase and clarifications sought.
- The Business Analysts document the requirement based on their discussion with the customer.
- Will the new banking application be used in more than one country?
- Do we have to support multiple languages?
- How many users are expected to use the application? etc

Water fall model-example-Banking Application

- The architect and senior members of the team work on the software architecture, high level and low level design for the project.
- It is decided that the banking application needs to have redundant backup and failover capabilities such that system is accessible at all times.

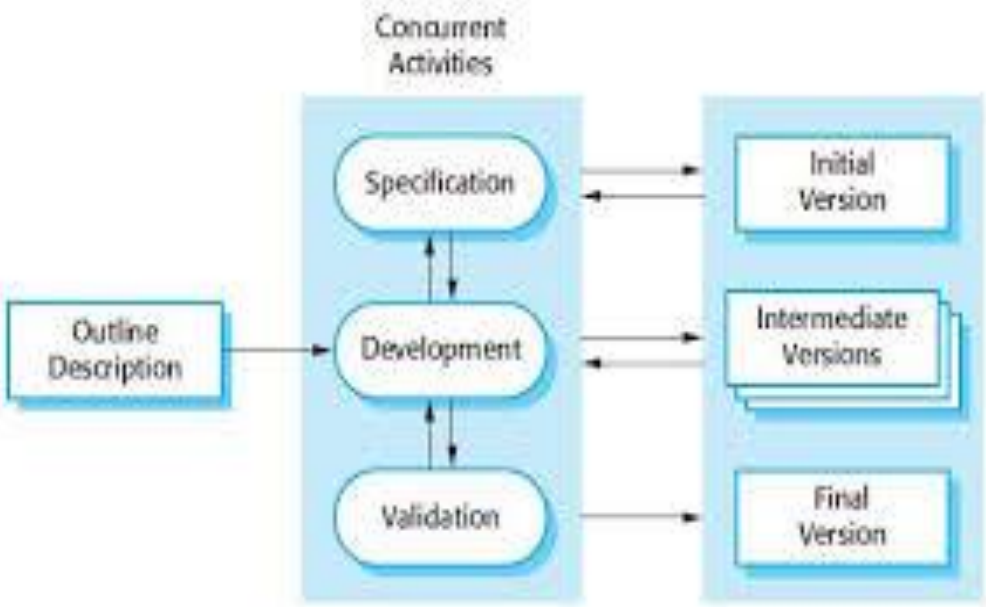
Water fall model-example-Banking Application

- The development team works on coding the project.
- Security checks will be implemented
- Reviews, static analysis will be done

Water fall model-example-Banking Application

- The testing team tests the complete application and identifies any defects in the application.
- Some of the high level activities of deployment include installing the OS on the servers, installing security patches, hardening the servers, installing web servers and application servers, installing the database etc.

Incremental development



Incremental development

- Incremental development is based on the idea of developing an initial implementation, exposing this to user comment and evolving it through several versions until an adequate system has been developed

Incremental development benefits

- The cost of accommodating changing customer requirements is reduced.
- It is easier to get customer feedback on the development work that has been done.
- More rapid delivery and deployment of useful software to the customer is possible.

Incremental development problems

- The process is not visible
- System structure tends to degrade as new increments are added

Incremental and Iterative development

- Suppose to add a new login functionality to a website, and is decided to develop this using agile methodology, working in two-week delivery cycles (iterations).

Incremental and Iterative development

- **First iteration:**
- The *minimum* working version of the login functionality worth delivering to users would be
- Create a web page where the users will see the login form
- Add a login form with just two fields (username and password) and a login button, i.e. just a normal HTML form with ***no*** styling or validation.
- Create a “welcome” page so when a users log in, they will see a message.

This basic login feature went through design, development, and testing and delivered at the end of the iteration.

Incremental and Iterative development

- **Second iteration:**
- In next iteration, we want to enhance the login functionality of what was built in the last iteration. We may decide to do
- Build validation rules around input parameters
- Add some CSS so the login form looks pretty
- Display a message when user tries to login with invalid credentials

- Now new functionality is added and enhanced existing functionality. In other words, we have *incremented* the existing login functionality, in this iteration.

Incremental and Iterative development

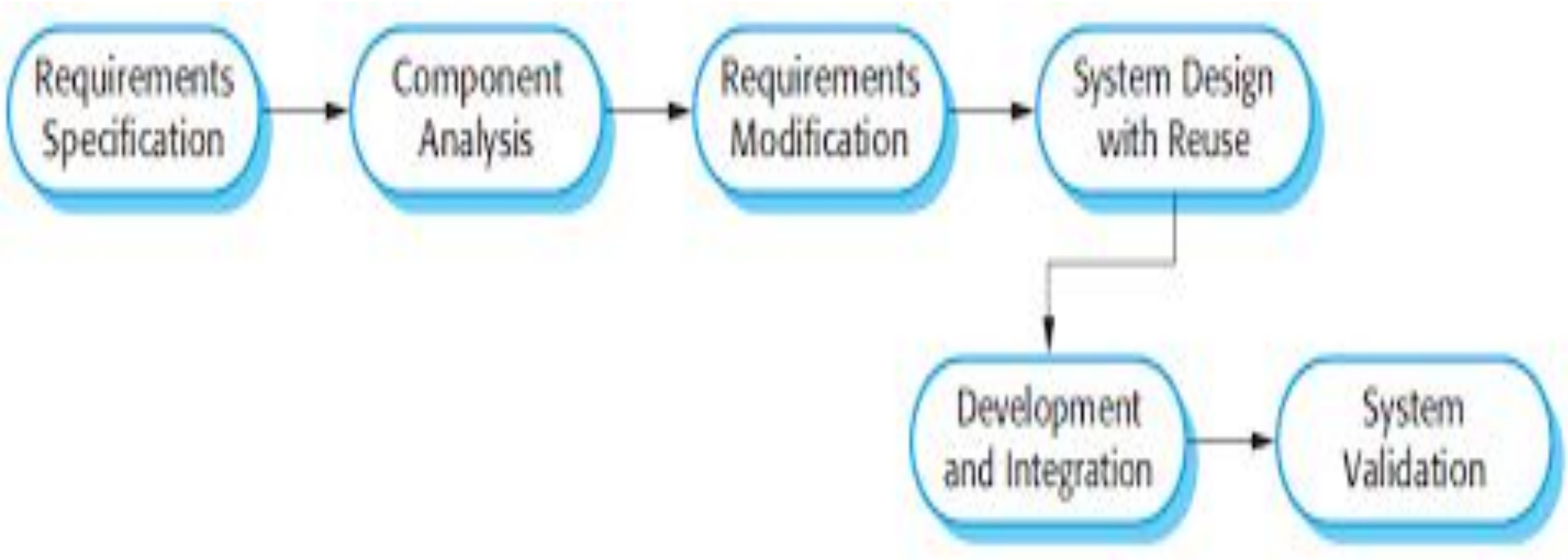
- **Third iteration:**
- In iteration three, we can yet again increment our login functionality, by adding
- Forgotten password functionality
- “Remember me” checkbox
- Redirection mechanism to redirect to appropriate pages when users log in.

- in each iteration, the login functionality is incremented by adding new useful features for the users.

Reuse- Oriented Software Engineering

- In the majority of software projects, there is some software reuse.
- Based on software reuse where systems are integrated from existing components or COTS (Commercial-off-theshelf) systems.
- Reused elements may be configured to adapt their behaviour and functionality to a user's requirements.

Reuse- Oriented Software Engineering



Types of reusable software

- Standalone applications
- Packages
- Web services

Advantages and disadvantages

- Reduced costs and risks as less software is developed from scratch
- Faster delivery and deployment of system
- requirements compromises are inevitable so system may not meet real needs of users
- Loss of control over evolution of reused system elements

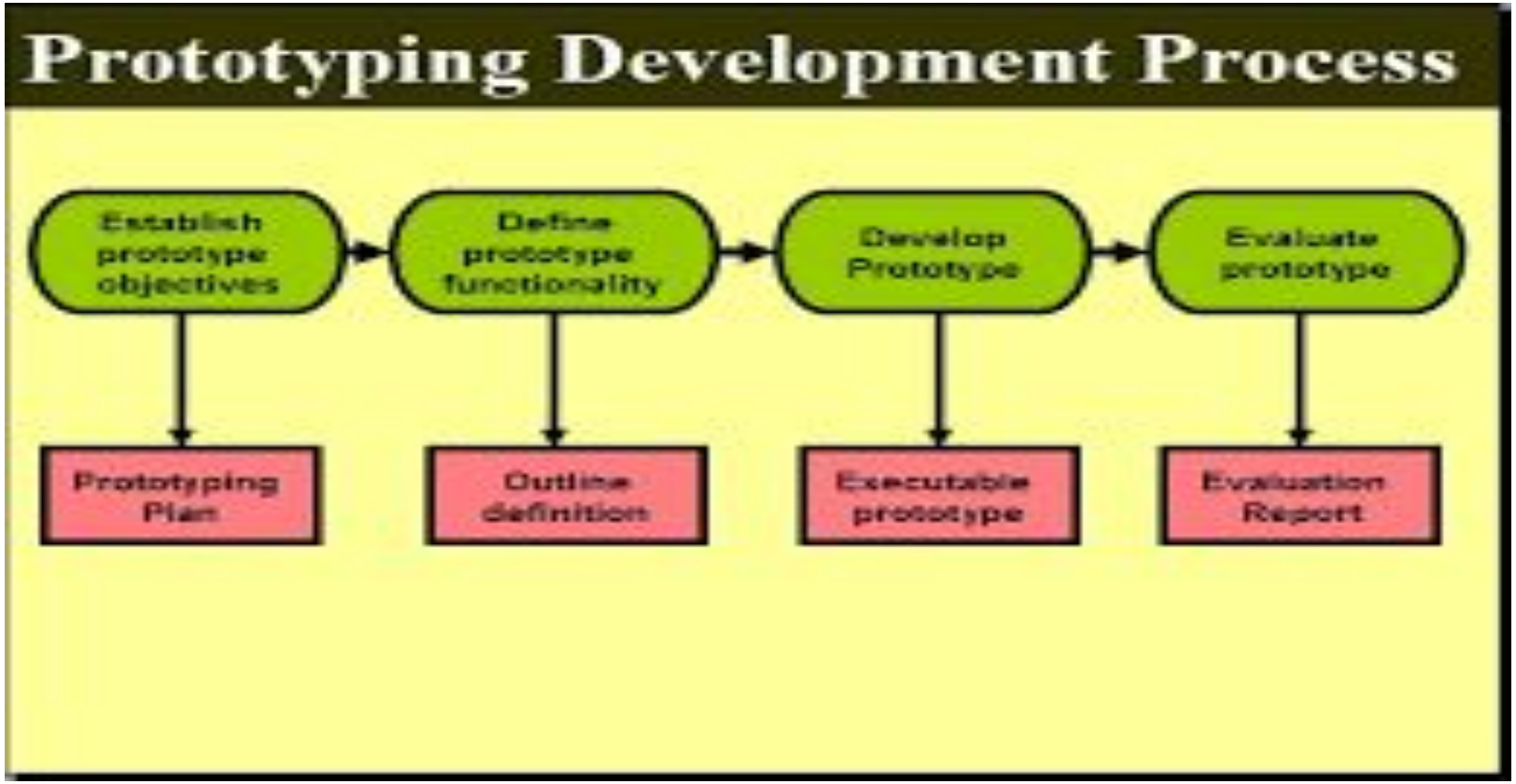
Coping with change

- Change is inevitable in all large software projects.
- Business changes lead to new and changed system requirements
- New technologies open up new possibilities for improving Implementations
- Changing platforms require application changes
- Change leads to rework so the costs of change include both rework as well as the costs of implementing new functionality

Reducing the costs of rework

- Change anticipation-prototype
- Change tolerance-incremental development

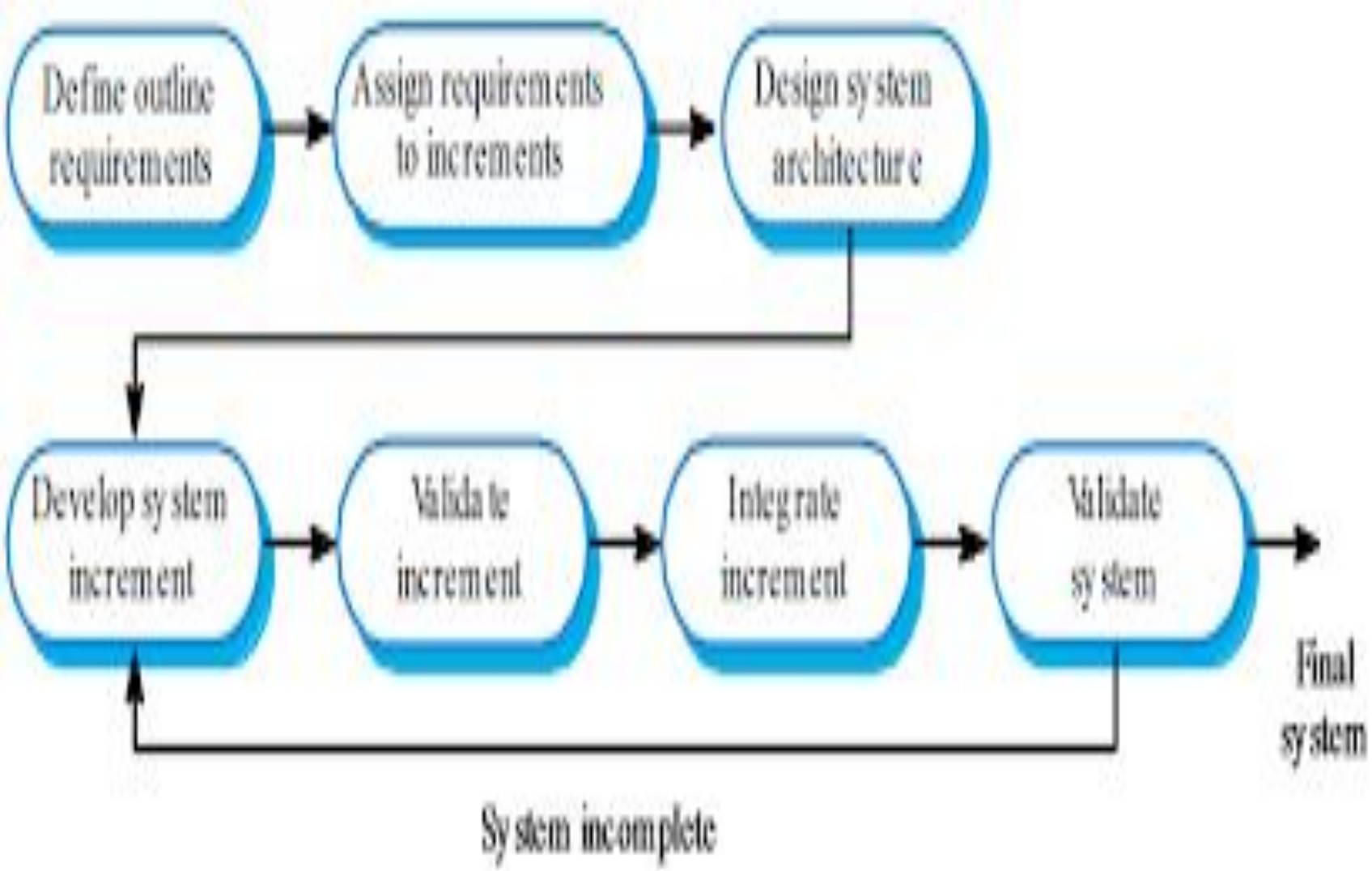
The process of prototype development



Throw-away prototypes

- Prototypes should be discarded after development as they are not a good basis for a production system
- It may be impossible to tune the system to meet non-functional requirements
- Prototypes are normally undocumented;

Incremental delivery



Process improvement

- way of enhancing the quality of their software, reducing costs or accelerating their development processes.
- Process improvement means understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time.

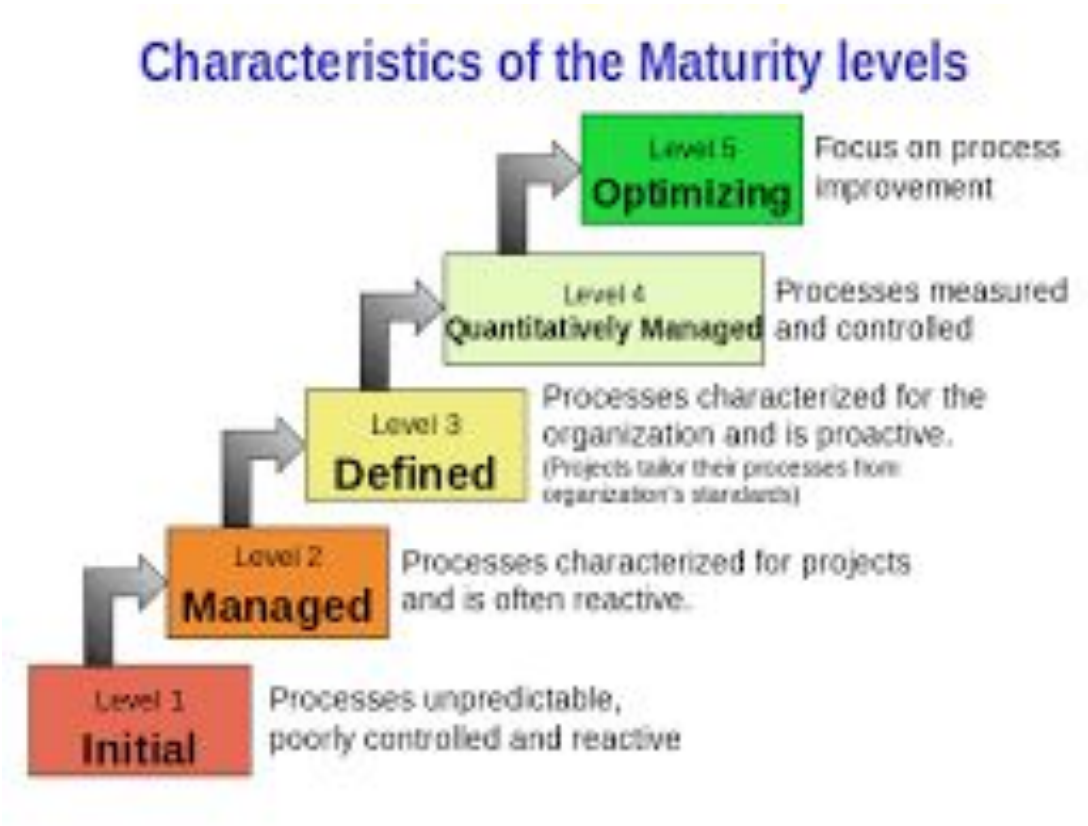
Process improvement activities

- Process measurement
- Process analysis-process maps are developed
- Process change

Process metrics

- Time taken for process activities to be completed
- Resources required for processes or activities
- Number of occurrences of a particular event

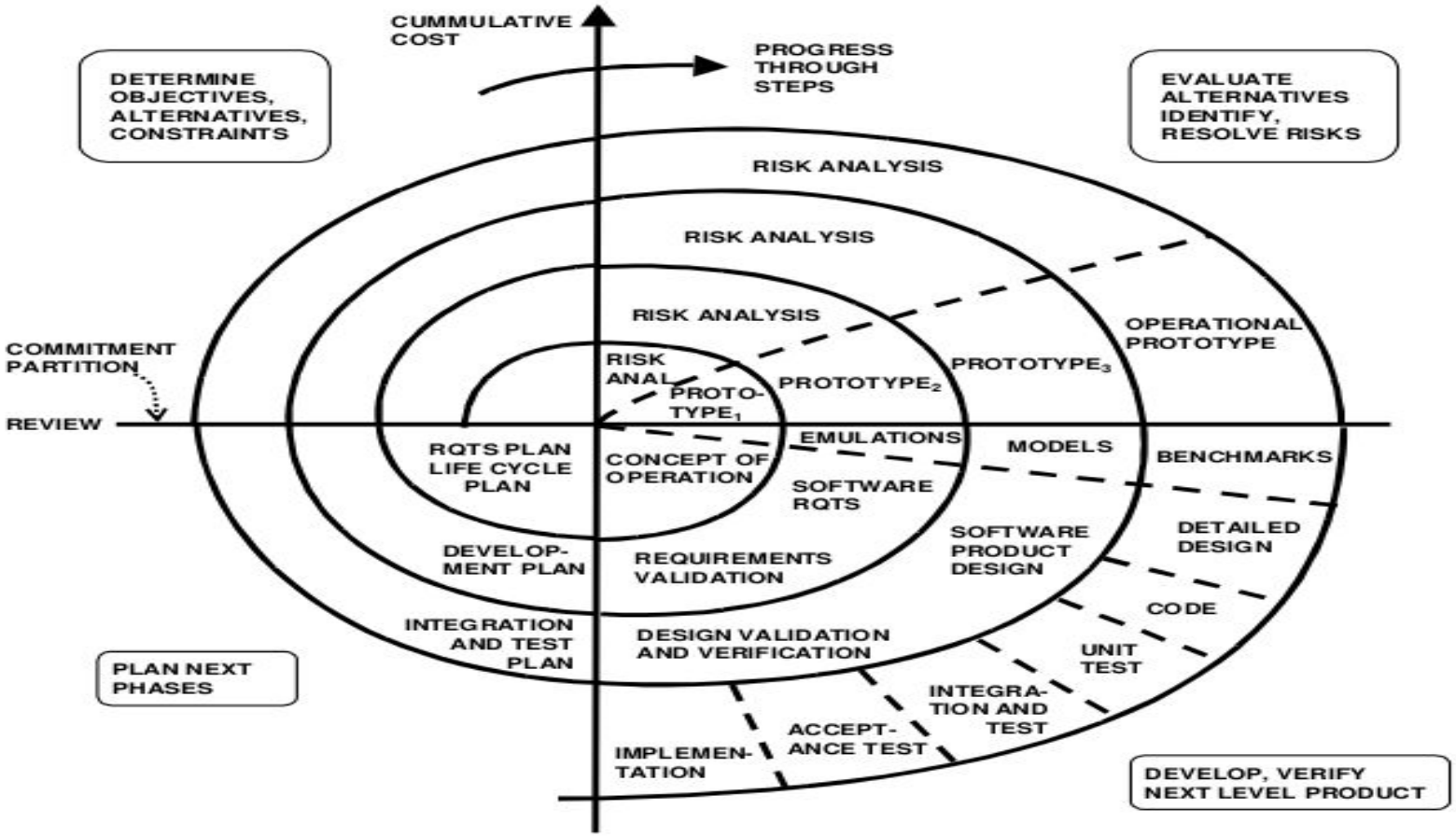
Capability maturity levels



CMM Level	Focus	Key Process Areas
1. Initial	Competent People	NO KPA'S
2. Repeatable	Project Management	Software Project Planning software Configuration Management
3. Defined	Definition of Processes	Process definition Training Program Peer reviews
4. Managed	Product and Process quality	Quantitiative Process Metrics Software Quality Management
5. Optimizing	Continuous Process improvement	Defect Prevention Process change management Technology change management

The focus of each SEI CMM level and the Corresponding Key process areas.

Boehm's spiral model



Boehm's spiral model

- Combination of iterative and waterfall development
- High emphasis on risk analysis
- software process is represented as a spiral, rather than a sequence of activities
- Each loop in the spiral represents a phase of the software process.
- innermost loop might be concerned with system feasibility, the next loop with requirements definition, the next loop with system design,

Boehm's spiral model

- Each loop in the spiral is split into four sectors:
- *Objective setting*
- *Risk assessment and reduction*
- *Development and validation*
- *Planning*

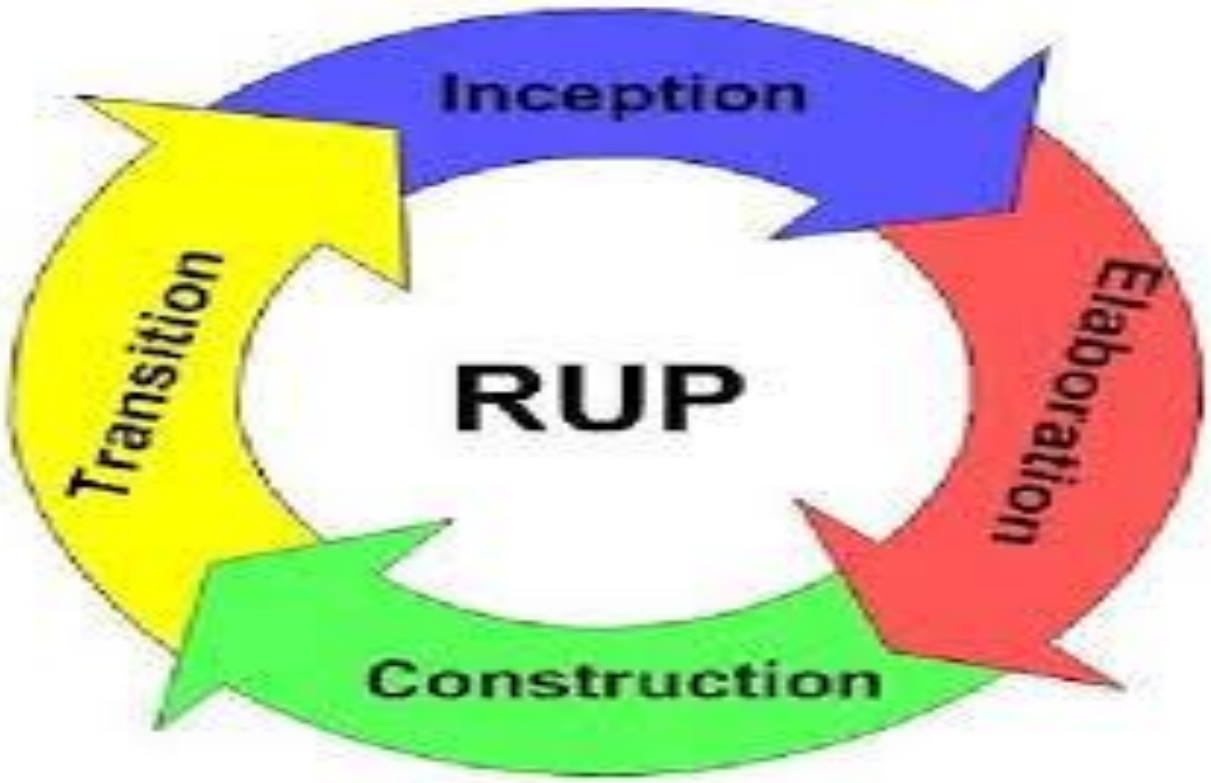
Boehm's spiral model applications

- When project is large.
- When releases are required to be frequent.
- When creation of a prototype is applicable.
- When risk and costs evaluation is important.
- For medium to high-risk projects.
- When requirements are unclear and complex.
- When changes may require at any time.

The Rational Unified Process

- Derived from work on UML
- Created by Rational Software Corporation
- RUP is normally described from three perspectives
- *A dynamic perspective*, which shows the phases of the model over time.
- *A static perspective*, which shows the process activities that are enacted.
- *A practice perspective*, which suggests good practices to be used during the process

The Rational Unified Process



The Rational Unified Process

RUP - Lifecycle Phases

Inception

- vision document
  - scope the system
  - identify major players
  - risk, cost etc

Elaboration

- risk identification
- problem domain
- analysis & architecture

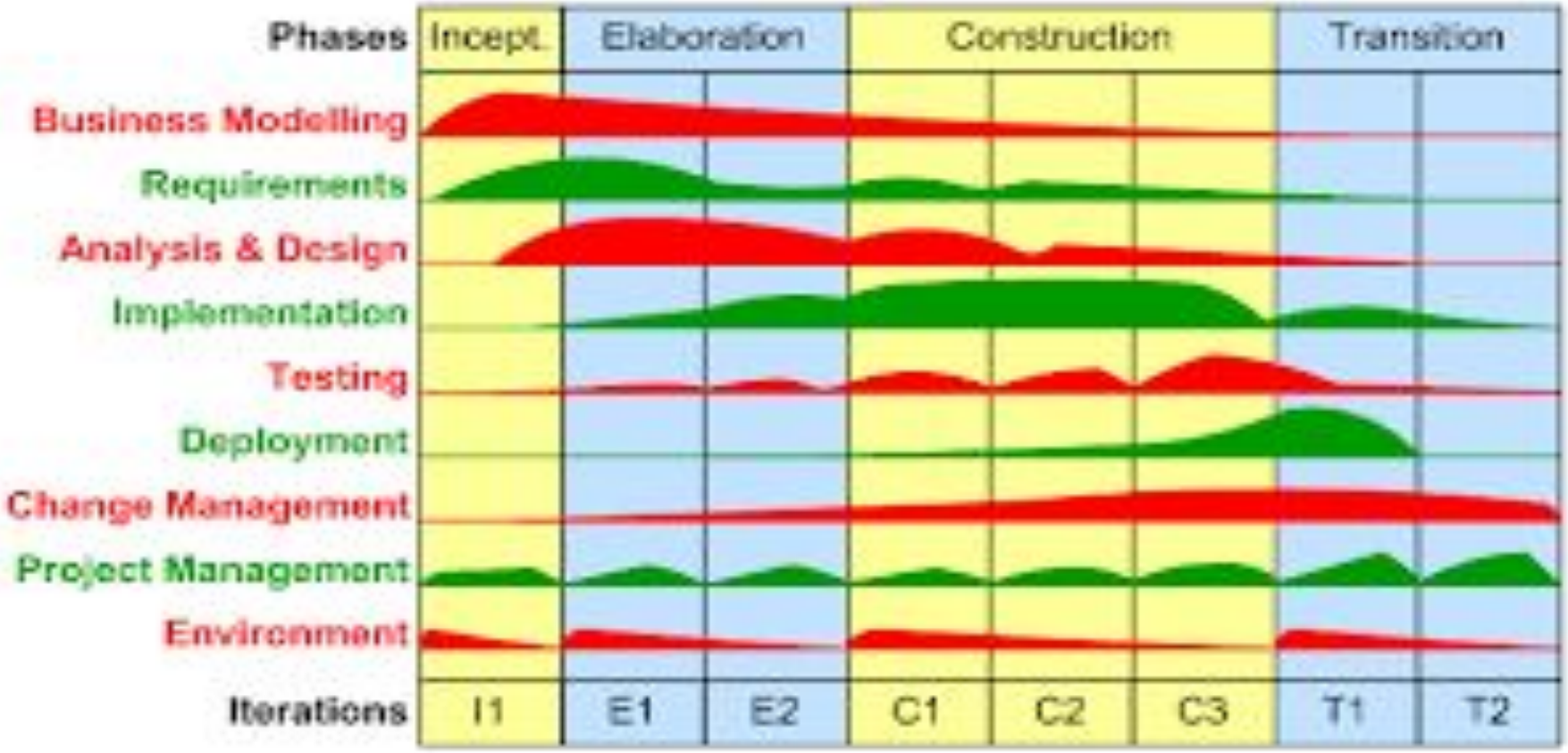
Construction

- build the software
- can be broken down into iterations

Transition

- transition from development to production

workflows


Phases
Incept.
Elaboration
Construction
Transition
Business Modelling
Requirements
Analysis & Design
Implementation
Testing
Deployment
Change Management
Project Management
Environment
Iterations
I1
E1
E2
C1
C2
C3
T1
T2

Best practices of RUP

- *Develop software iteratively*
- *Manage requirements*
- *Use component-based architectures*
- *Visually model software*
- *Verify software quality*
- *Control changes to software*

Agile Software Development

- Introduction to agile methods
- Agile development techniques
- Agile project management
- Scaling agile methods.

Rapid software development

- Rapid development and delivery is now often the most important requirement for software systems.
- Businesses operate in a fast –changing requirement and it is practically impossible to produce a set of stable software requirements.
- Software has to evolve quickly to reflect changing business needs
- Specification, design and implementation are inter-leaved
- System is developed as a series of versions with stakeholders involved in version evaluation.
- User interfaces are often developed using an IDE and graphical toolset.

Rapid software development

- Focus on the code rather than the design
- Are based on an iterative approach to software development
- Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- The aim of agile methods is to reduce overheads in the software process (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.

Rapid software development

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Methodology

The principles of agile methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Agile method applicability

- Product development where a software company is developing a small or medium-sized product for sale.
- Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.
- Because of their focus on small, tightly-integrated teams, there are problems in scaling agile methods to large systems.

Problems with agile methods

- It can be difficult to keep the interest of customers who are involved in the process.
- Team members may be unsuited to the intense involvement that characterises agile methods.
- Prioritizing changes can be difficult where there are multiple stakeholders.
- Maintaining simplicity requires extra work
- Contracts may be a problem as with other approaches to iterative development

Agile methods and software maintenance

- if agile methods are to be successful, they have to support maintenance as well as original development.
- Two key issues:
- Are systems that are developed using an agile approach maintainable, given the emphasis in the development process of minimizing formal documentation?
- Can agile methods be used effectively for evolving a system in response to customer change requests?
- Problems may arise if original development team cannot be maintained.

Technical, human, organizational issues

- Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on:
- Is it important to have a very detailed specification and design before moving to implementation? If so, you probably need to use a plan-driven approach.
- Is an incremental delivery strategy, where you deliver the software to customers and get rapid feedback from them, realistic? If so, consider using agile methods.

Technical, human, organizational issues

- How large is the system that is being developed? Agile methods are most effective when the system can be developed with a small co-located team who can communicate informally. This may not be possible for large systems that require larger development teams so a plan-driven approach may have to be used.
- What type of system is being developed?
- Plan-driven approaches may be required for systems that require a lot of analysis before implementation

Technical, human, organizational issues

- What is the expected system lifetime?
- Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team.
- What technologies are available to support system development?
- Agile methods rely on good tools to keep track of an evolving design
- How is the development team organized?
- If the development team is distributed or if part of the development is being outsourced, then you may need to develop design documents to communicate across the development teams

Technical, human, organizational issues

- Are there cultural or organizational issues that may affect the system development?
- Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering.
- How good are the designers and programmers in the development team?
- Is the system subject to external regulation?

Extreme programming

- Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.
- New versions may be built several times per day;
- Increments are delivered to customers every 2 weeks;
- All tests must be run for every build and the build is only accepted if tests run successfully.

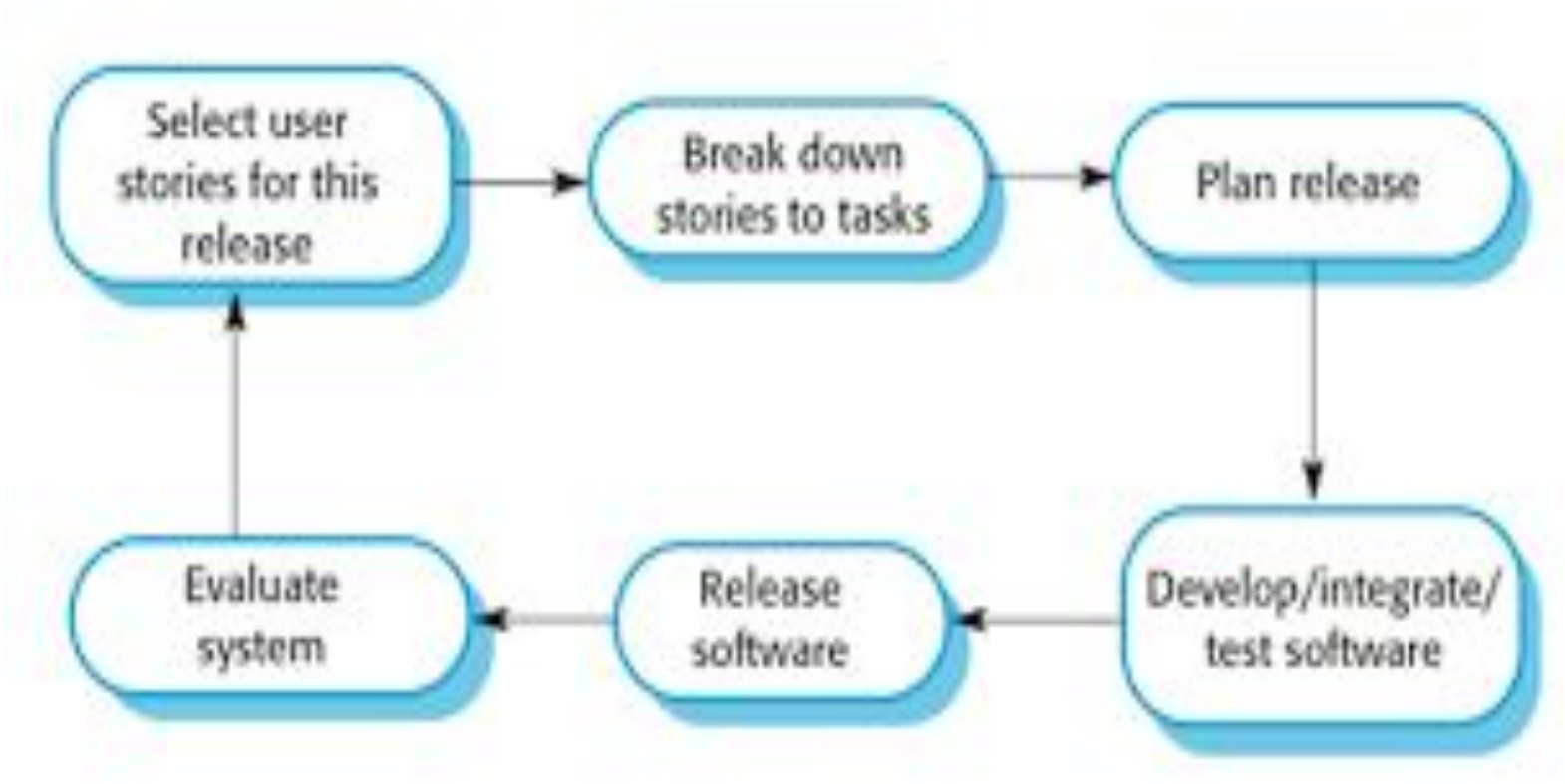
XP and agile principles

- Incremental development is supported through small,frequent system releases.
- Customer involvement means full-time customer engagement with the team.
- People not process through pair programming, collective ownership and a process that avoids long working hours.
- Change supported through regular system releases.
- Maintaining simplicity through constant refactoring of code.

Extreme programming

- Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.
- New versions may be built several times per day;
- Increments are delivered to customers every 2 weeks;
- All tests must be run for every build and the build is only accepted if tests run successfully.

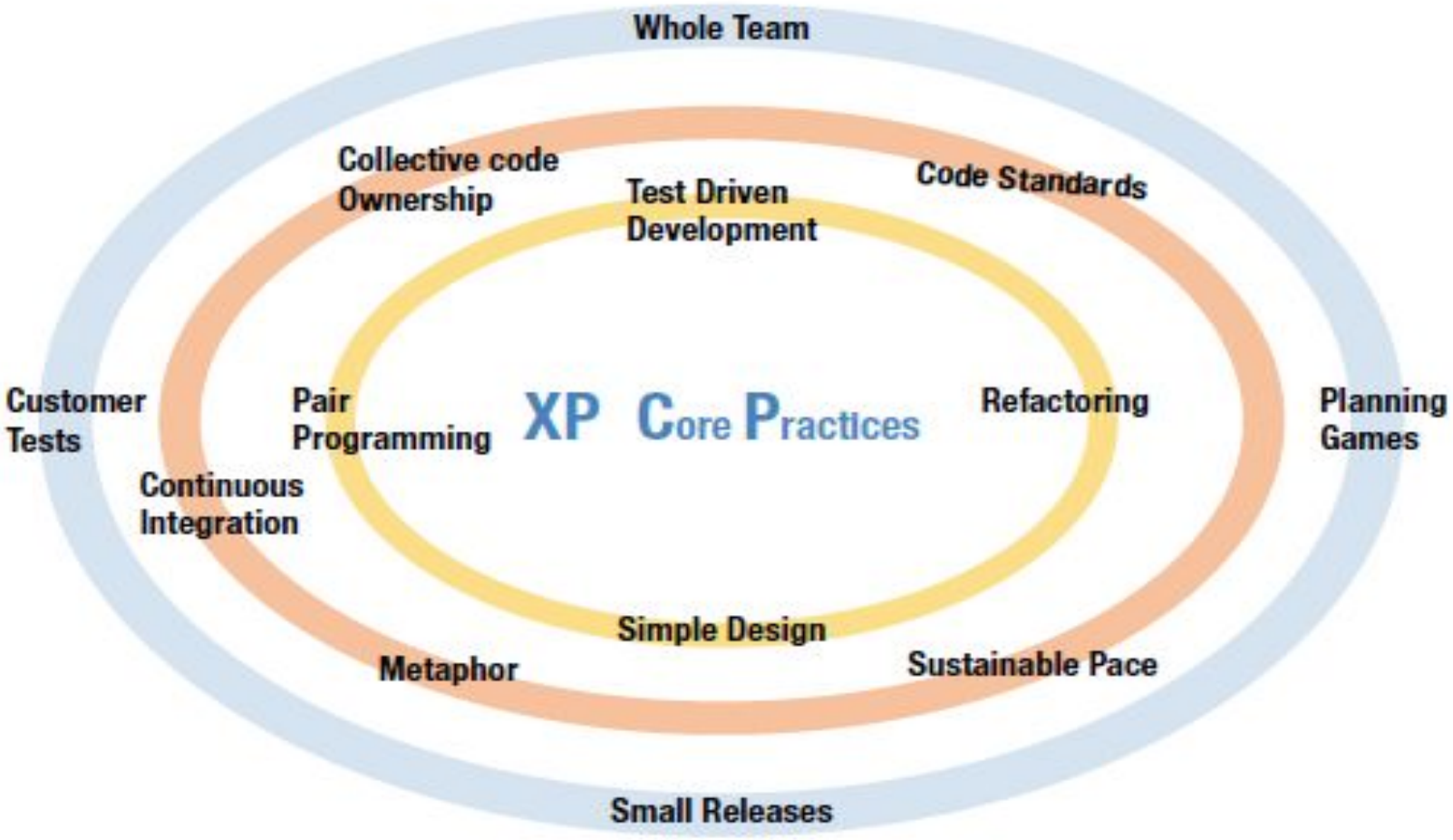
Extreme programming



User stories

Title:	**Priority:**	**Estimate:**
As a *<type of user>* **I want to** *<perform some task>* **so that I can** *<achieve some goal>*		
Acceptance criteria **Given** *<some context>* **When** *<some action is carried out>* **Then** *<a set of observable outcomes should occur>*		

Extreme programming practices



XP and change

- Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated.
- Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.

Refactoring

- Programming team look for possible software improvements and make these improvements even where there is no immediate need for them.
- This improves the understandability of the software and so reduces the need for documentation.
- Changes are easier to make because the code is well structured and clear.
- However, some changes requires architecture refactoring and this is much more expensive.

Testing in XP

- XP testing features:
- Test-first development.
- Incremental test development from scenarios.
- User involvement in test development and validation.
- Automated test harnesses are used to run all component tests each time that a new release is built.

Test-first development

- Writing tests before code clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
- All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors

Customer involvement

- The role of the customer in the testing process is to help develop acceptance tests for the stories that are to be implemented in the next release of the system.
- The customer who is part of the team writes tests as development proceeds. All new code is therefore validated to ensure that it is what the customer needs.
- However, people adopting the customer role have limited time available and so cannot work full-time with the development team. They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.

XP testing difficulties

- Writing incomplete tests
- Difficulty in writing test cases
- Checking completeness of the tests

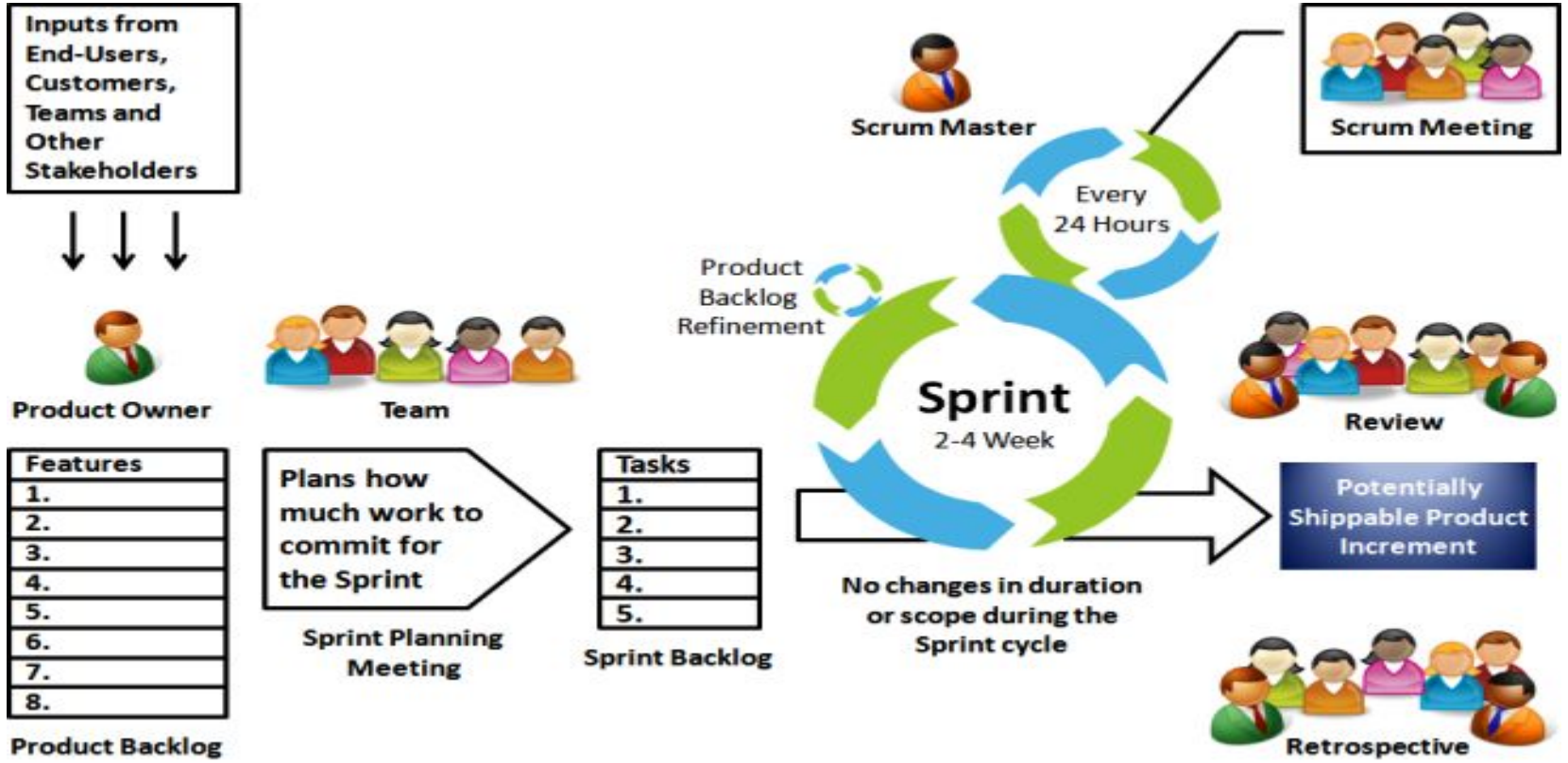
Pair programming

- In XP, programmers work in pairs, sitting together to develop code.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from this.
- Measurements suggest that development productivity with pair programming is similar to that of two people working independently

Scrum

- The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices.
- There are three phases in Scrum.
- The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
- This is followed by a series of sprint cycles, where each cycle develops an increment of the system.
- The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

The Scrum process



The Sprint cycle

- Sprints are fixed length, normally 2–4 weeks. They correspond to the development of a release of the system in XP.
- The starting point for planning is the product backlog, which is the list of work to be done on the project.
- The selection phase involves all of the project team who work with the customer to select the features and functionality to be developed during the sprint.

The Sprint cycle

- Once these are agreed, the team organize themselves to develop the software.During this stage the team is isolated from the customer and the organization, with all communications channelled through the so-called ‘Scrum master’.
- The role of the Scrum master is to protect the development team from external distractions.
- At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Teamwork in Scrum

- The ‘Scrum master’ is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.
- This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them. `

Scrum benefits

- The product is broken down into a set of manageable and understandable chunks.
- Unstable requirements do not hold up progress.
- The whole team have visibility of everything and consequently team communication is improved.
- Customers see on-time delivery of increments and gain feedback on how the product works.
- Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

Scaling agile methods

- Agile methods have proved to be successful for small and medium sized projects that can be developed by a small co-located team.
- It is sometimes argued that the success of these methods comes because of improved communications which is possible when everyone is working together.
- Scaling up agile methods involves changing these to cope with larger, longer projects where there are multiple development teams, perhaps working in different locations.

Large systems development

- Large systems are usually collections of separate, communicating systems, where separate teams develop each system. Frequently, these teams are working in different places, sometimes in different time zones.
- Large systems are ‘brownfield systems’, that is they include and interact with a number of existing systems. Many of the system requirements are concerned with this interaction and so don’t really lend themselves to flexibility and incremental development.
- Where several systems are integrated to create a system, a significant fraction of the development is concerned with system configuration rather than original code development.

Large systems development

- Large systems and their development processes are often constrained by external rules and regulations limiting the way that they can be developed.
- Large systems have a long procurement and development time. It is difficult to maintain coherent teams who know about the system over that period as, inevitably, people move on to other jobs and projects.
- Large systems usually have a diverse set of stakeholders.It is practically impossible to involve all of these different stakeholders in the development process.

Scaling out and scaling up

- ‘Scaling up’ is concerned with using agile methods for developing large software systems that cannot be developed by a small team.
- ‘Scaling out’ is concerned with how agile methods can be introduced across a large organization with many years of software development experience.
- When scaling agile methods it is essential to maintain agile fundamentals
- Flexible planning, frequent system releases, continuous integration, test-driven development and good team communications.

Scaling out and scaling up

- For large systems development, it is not possible to focus only on the code of the system. You need to do more up-front design and system documentation
- Cross-team communication mechanisms have to be designed and used. This should involve regular phone and video conferences between team members and frequent, short electronic meetings where teams update each other on progress.
- Continuous integration, where the whole system is built every time any developer checks in a change, is practically impossible. However, it is essential to maintain frequent system builds and regular releases of the system

Scaling out to large companies

- Project managers who do not have experience of agile methods may be reluctant to accept the risk of a new approach.
- Large organizations often have quality procedures and standards that all projects are expected to follow and, because of their bureaucratic nature, these are likely to be incompatible with agile methods.
- Agile methods seem to work best when team members have a relatively high skill level. However, within large organizations, there are likely to be a wide range of skills and abilities.
- There may be cultural resistance to agile methods, especially in those organizations that have a long history of using conventional systems engineering processes.